

2026-05-14 — ConverseQuad3D p=0.6 overflow/NaN training fix

Symptom

During the `ConverseQuad3D-v0` aerial sweep, SAC at `p=0.6` crashed after early rollouts with overflow warnings from NumPy/SB3 and then a PyTorch failure while constructing the SAC policy's `Normal(mean_actions, action_std)` distribution. The logs showed enormous episode costs/rewards, `inf` oracle Bellman residuals, `VecNormalize` running-stat warnings, and finally invalid actor outputs.

Root cause

This was an environment numerical-domain failure rather than a primary Torch bug. The learner acts in raw actuator coordinates `nu`, but the benchmark uses the cubic virtual action

```
u_i = phi_i(nu_i) = nu_i + eta * nu_i^3, eta = 0.1.
```

With the default raw action box `|nu_i| <= 10`, full-range SAC exploration can produce virtual actions near `|u_i| = 110`. In the harder `p=0.6` setting those commands can drive the state to extremely large values. The old environment only sanitized non-finite `s_next` after the fact and still allowed huge finite states to be cast to `float32`, overflowing observations and corrupting `VecNormalize` statistics. SAC then received `nan` normalized observations and eventually produced invalid Gaussian action-distribution parameters.

A quick stress reproduction with deterministic random full-box actions showed `p=0.6` reaching the `float32` overflow region, while easier `p=0.2/p=0.4` rollouts remained finite for the same horizon.

Fix

Implemented in the canonical aerial files under `Code mode 2/Code/`:

- `converse_aerial_env.py`
 - Added finite defaults: `obs_clip=1e6, state_limit=1e6, cost_limit=1e12, terminate_on_unhealthy=True, info_clip=1e30`.
 - Sanitizes observations before `float32` conversion.
 - Detects non-finite/too-large next states and costs.
 - Terminates unhealthy transitions with a finite penalty instead of returning overflowing observations/rewards.
 - Clips/sanitizes info diagnostics and oracle-action casts so TensorBoard and SB3 never ingest `nan/inf` from pathological rollouts.

- `train_baselines_aerial_gpu.py`
 - Threads safety parameters through CLI/env kwargs and `meta.json`.
 - Logs safety diagnostics to TensorBoard:
`train/unhealthy_transition`, `train/terminated_by_safety`,
`train/state_norm_unclipped`, and `train/stage_cost_unclipped`.
 - Starts TensorBoard by default when available, watching the sweep log root.
- `evaluate_aerial.py`, YAML configs, and benchmark spec
 - Threaded the same safety parameters into evaluation/config metadata.
- `tests/test_aerial_sanity.py`
 - Added a $p=0.6$ random full-action-box regression test requiring finite observations, rewards, and diagnostics until safety termination/truncation.

The certified converse family/oracle equations were not changed; the guard only keeps learner-induced pathological rollouts inside SB3's finite numerical domain.

Validation performed

```
source .venv/bin/activate

# Manual equivalent of pytest because pytest was not installed in the
# venv.
python - <<'PY'
import importlib.util, sys
sys.path.insert(0, 'Code mode 2/Code')
spec =
importlib.util.spec_from_file_location('test_aerial_sanity_manual',
'Code mode 2/Code/tests/test_aerial_sanity.py')
mod = importlib.util.module_from_spec(spec)
spec.loader.exec_module(mod)
for name in sorted(n for n in dir(mod) if n.startswith('test_')):
    getattr(mod, name)()
PY

python Code/validate_aerial_fixture.py --num_states 50

python Code/train_baselines_aerial_gpu.py \
  --algo SAC --total_steps 1024 --learning_starts 64 \
  --train_freq 16 --gradient_steps 16 --batch_size 64 \
  --buffer_size 5000 --eval_every 512 --seed 0 --p 0.6 \
  --sigma_noise 0.01 --device cpu --num_envs 1 --normalize \
  --log_root aerial_results/smoke_fix \
  --launch_tensorboard true --tensorboard_port 6006 \
  --tensorboard_logdir aerial_results/smoke_fix
```

Results:

- Manual aerial sanity tests passed.

- `validate_aerial_fixture.py --num_states 50` passed strict mathematical checks (S orthogonality, energy identity, Bellman residual, `phi_inv`).
- SAC $p=0.6$ smoke training completed without the previous NaN/Normal crash and started TensorBoard at <http://127.0.0.1:6006/>.